

Music Decoder

Assessment 2: Microcontroller Project

Task Summary

Your task is to program the QUTy development board to produce music and animations by interpreting Base64 encoded data. This program will interface with the piezo buzzer and two-digit 7-segment display, and receive inputs from the potentiometer, pushbuttons, and UART.

Submission and Due Date

Submit your code via the Gradescope portal by Friday 23 October 2026, 11:59pm. The automatic 48-hour extension applies.

Contents

1	Required Functionality	1
2	Operating States	1
3	Terminology	1
4	The Data	2
4.1	Supplied Data	2
4.2	Reading from Program Space	2
4.3	Base64 Decoding	2
4.4	LFSR Descrambling	3
4.5	Music Instruction Format	4
4.5.1	Duration	4
4.5.2	Octave and Note	4
4.5.3	Display Byte	5
4.6	Tracks	5
4.7	Test Message	5
5	Track Selection	5
5.1	Multiplier Selection	5
6	Music Player	6
6.1	Playing	6
6.2	Paused	6
6.3	Track Seeking	7
7	Serial Interface	7

7.1	Message Format	7
7.2	Commands	7
7.3	Responses	8
7.4	Examples	8
8	Hardware Configuration	9
8.1	Pushbuttons	9
8.2	Potentiometer	9
8.3	7-Segment Display	9
8.4	Buzzer	9
A	Segment Patterns	9

1 Required Functionality

Your program must:

1. decode the supplied Base64 encoded data;
2. descramble the decoded data using a linear-feedback shift register;
3. interpret three-byte music instructions;
4. play notes through the buzzer using pulse-width modulation;
5. display custom patterns on the 7-segment display;
6. support *track selection*, *multiplier selection*, *playback*, *pause*, *step*, *stop*, and *seeking*; and
7. support a serial command parser via UART.

2 Operating States

Your program must support three states as specified in [Table 1](#).

State	Behaviour
TRACK_SELECT	Accept a choice of track and multiplier.
PLAYING	Execute instructions automatically in sequence.
PAUSED	Keep the current instruction active, allowing stepping, seeking, re-summing, and stopping.

Table 1: Program states.

3 Terminology

The data supplied to you passes through several stages before it can be played. [Table 2](#) defines terminology used in this document.

Term	Definition
Encoded data	The Base64 encoded characters supplied in your header file.
Decoded data	The bytes recovered by decoding the Base64 encoded data.
Music data	The bytes that make up instructions. ¹
Instruction	Three consecutive music bytes specifying a duration, a pitch, and a display byte.
Terminating instruction	An instruction consisting of three zero bytes, marking the end of a track.
Track	A sequence of consecutive instructions.
Instruction number	The position of an instruction in the decoded data, starting from zero.
Seek value	The eight-bit ADC result that selects an instruction in Track 4.
Multiplier	The value scaling the playback duration of every instruction.
Display byte	The literal byte written to the display, driving one digit and its segments.
LFSR state	The 32-bit value held by the linear-feedback shift register.

Table 2: Terminology used in this document.

¹Only Track 4 is scrambled, so music data and decoded data are the same for Tracks 1–3.

4 The Data

4.1 Supplied Data

You have been provided `data.c` and `data.h`, containing the data your program needs to recover and play music. The following symbols are declared in `data.h`:

- `encoded_data[]`: the Base64 encoded data;
- `test_message[]`: a Base64 encoded string used to check your decoder and descrambler; and
- `track_start[4]`: the instruction number at which each track begins.

Use `#include "data.h"` to access these variables, but do not modify the contents of either file.

4.2 Reading from Program Space

As `encoded_data` is too large to hold in SRAM, the `PROGMEM` attribute in `data.c` places it in flash, alongside your compiled program. Program memory and SRAM occupy separate address spaces on the ATtiny1626, so you must use the functions declared in `<avr/pgmspace.h>` to dereference a pointer into `encoded_data`.

Listing 1 provides two methods for reading four encoded characters into a buffer in SRAM.

```
#include <avr/pgmspace.h>

const char encoded_data[] PROGMEM = "Cats";
char buffer[4];

// reading one character at a time
for (uint8_t i = 0; i < 4; i++) {
    buffer[i] = pgm_read_byte(&encoded_data[i]);
}

// reading four characters at a time
memcpy_P(buffer, encoded_data, 4);
```

Listing 1: Reading four characters from program memory with `pgm_read_byte` and `memcpy_P`.

4.3 Base64 Decoding

Base64 represents binary data using a set of 64 printable characters, so that arbitrary bytes can be stored as ordinary text. Encoding is necessary because instruction bytes may take any value from `0x00` to `0xFF`, including special control characters that cannot be printed as text, and `0x00` itself, which would otherwise be interpreted as the end of a string.

Base64 characters carry six bits of information² so that four Base64 characters decode to a sequence of three bytes. The data supplied to you is always a multiple of three bytes in length, so every group of four characters is complete. Table 3 gives the value of each character in this encoding.

²Six bits are required to represent 64 unique characters.

Characters	Decoded value
A–Z	0–25
a–z	26–51
0–9	52–61
+	62
/	63

Table 3: Base64 characters and corresponding decoded values.

In a sequence of four Base64 characters, each character is replaced by its six-bit value, and the resulting 24 bits are divided into three bytes. They are decoded together because 24 bits is the smallest quantity that divides evenly into both six-bit characters and eight-bit bytes. No decoded byte is aligned to a single character, so your program must always decode four characters at a time. Figure 1 decodes the characters **Cats**.

Encoded data	C	a	t	s																		
Decoded value	2	26	45	44																		
Bits	00001001 10101011011011001100																					
Decoded data	0x09								0xAB								0x6C					

Figure 1: Example of decoding the four Base64 characters **Cats**.

The encoded data supplied in your header file is always valid. Characters received through the serial interface are not, so your program must verify them, as described in [Section 7](#).

4.4 LFSR Descrambling

The bytes belonging to Track 4 are scrambled with a linear-feedback shift register, so they must be descrambled before they can be interpreted as instructions. [Listing 2](#) descrambles a decoded byte.

```

bit      ← lsbit(lfsr)
lfsr     ← lfsr >> 1
if (bit = 1)
    lfsr  ← lfsr ⊕ MASK
music_byte ← lsbyte(lfsr) ⊕ decoded_byte

```

Listing 2: Descrambling algorithm for a single decoded byte.

In this algorithm:

MASK is the constant 0xE2026E6B,
lsbyte() returns the least significant byte,
lsbit() returns the least significant bit,
>> is the logical shift right operator, and
⊕ is the bitwise exclusive or operator.

Your program must:

1. initialise **lfsr** with your student number, interpreted as a 32-bit hexadecimal literal, so that **n12345678** gives **0x12345678**; and
2. apply [Listing 2](#) to the decoded bytes of Track 4 in order, because each LFSR state depends on the one before it.

4.5 Music Instruction Format

The music data is divided into instructions, each laid out as shown in [Figure 2](#).

Byte 0								Byte 1								Byte 2							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
DURATION								OCTAVE				NOTE				DISPLAY							

Figure 2: Music instruction bit fields.

Four Base64 characters decode to three bytes, which correspond to exactly one instruction. For Track 4, those decoded bytes must also be descrambled. Instruction n therefore begins at the character `encoded_data[4n]`.

4.5.1 Duration

Byte 0 of an instruction sets how long that instruction remains active before the next is applied. DURATION counts units of 5 ms, scaled by the multiplier described in [Section 5.1](#):

$$\text{playback duration} = \text{DURATION} \times 5 \text{ ms} \times \text{multiplier}.$$

Some representative values are shown in [Table 4](#).

DURATION	Playback duration
0x00 (0)	0 ms
0x01 (1)	5 ms
0x0A (10)	50 ms
0x64 (100)	500 ms
0xFF (255)	1275 ms

Table 4: Representative values of DURATION and corresponding playback durations, where the multiplier is 1.

4.5.2 Octave and Note

Byte 1 of an instruction selects the pitch produced by the buzzer. Its upper and lower nibbles correspond to OCTAVE and NOTE respectively, where NOTE selects one of the twelve notes of the chromatic scale according to [Table 5](#).

NOTE	Note name	NOTE	Note name
0x0	C	0x6	F#
0x1	C#	0x7	G
0x2	D	0x8	G#
0x3	D#	0x9	A
0x4	E	0xA	A#
0x5	F	0xB	B

Table 5: Values of NOTE and corresponding note names.

When OCTAVE is 0x0, or NOTE is greater than 0xB, the buzzer must be silent. Otherwise, the buzzer must produce the frequency:

$$f = 440 \times 2^{(\text{OCTAVE}-4) + (\text{NOTE}-9)/12} \text{ Hz}.$$

4.5.3 Display Byte

Byte 2 of an instruction holds the DISPLAY byte for one digit. Bit 7 selects the digit and is supplied as part of the value, so the byte must be written directly to the shift register without modification.

4.6 Tracks

The encoded data is divided into four tracks. Tracks 1–3 have been Base64 encoded, and must be decoded before they can be interpreted as instructions. Track 4 has been scrambled and then Base64 encoded, so it must be decoded and then descrambled.

Track T begins at the instruction number given by `track_start[T−1]`, and ends with a *terminating instruction* consisting of the three bytes `0x00 0x00 0x00`. When a terminating instruction is reached, playback must stop and the program must return to `TRACK_SELECT`.

4.7 Test Message

`test_message` is a null-terminated C string, scrambled and encoded in the same way as Track 4, beginning from the initial LFSR state. It is used by the serial command parser and must not be played, as it does not describe instructions.

5 Track Selection

`TRACK_SELECT` allows the user to choose which of the four tracks to play. In this state, the buzzer must be off, the left digit must display a zero, and the right digit must show the selected track number. Upon reset, the program must enter this state with Track 1 selected.

The pushbuttons active in this state allow the user to change the selection, set the multiplier, and start playback, as shown in Table 6.

Input	Action
Hold S1	Preview the multiplier chosen with the potentiometer.
Release S1	Apply the multiplier chosen by the last position of the potentiometer.
Press S2	Decrease the track selection.
Press S3	Increase the track selection.
Press S4	Start the selected track.

Table 6: Pushbuttons active while in the `TRACK_SELECT` state.

Pressing S2 while Track 1 is selected must have no effect. Likewise, pressing S3 while Track 4 is selected must have no effect.

Upon returning to `TRACK_SELECT` after a terminating instruction, the selection advances to the next track, except after Track 4, where Track 4 must remain selected. Stopping a track manually leaves the selection unchanged.

5.1 Multiplier Selection

Multiplier selection is available only in `TRACK_SELECT`. While S1 is held, the potentiometer selects a multiplier, shown as two hexadecimal digits on the display. Upon release, the program must apply that multiplier and return to displaying the selected track number.

The multiplier scales the playback duration of every instruction, so a larger multiplier slows playback. It is taken from the upper four bits of the ADC result, giving sixteen values from 1 to 16:

$$\text{multiplier} = \text{ADC_RESULT}[7:4] + 1.$$

Table 7 shows several examples.

ADC result	Upper nibble	Multiplier	Display	Playback duration
0x00	0x0 (0)	1	01	5 ms
0x40	0x4 (4)	5	05	25 ms
0x90	0x9 (9)	10	0A	50 ms
0xFF	0xF (15)	16	10	80 ms

Table 7: Multipliers and playback durations for example ADC results, where DURATION is 0x01.

Upon reset, the multiplier is set to 1. It is applied to every track until it is modified again.

6 Music Player

6.1 Playing

While in the PLAYING state, the music player executes instructions one after another. For each instruction, the buzzer and display must produce their corresponding outputs, as described in Section 4.5. When the playback duration expires, the next instruction must be applied immediately. Any pushbutton or serial inputs received at the same time must be actioned immediately after the next instruction is applied. These inputs must not be discarded.

The pushbuttons active during playback, shown in Table 8, allow the user to interrupt the track, either temporarily or completely.

Input	Action
Press S3	Stop and return to TRACK_SELECT.
Press S4	Pause on the current instruction.

Table 8: Pushbuttons active while in the PLAYING state.

6.2 Paused

While the music player is in the PAUSED state, the current note and display byte must remain active, and the instruction timer must not advance. Resuming plays the current instruction for its full duration again, rather than the remaining time when it was paused. From here the user may step through the track one instruction at a time, seek to a new position, resume, or stop, using the pushbutton inputs shown in Table 9.

Input	Action
Hold S1	Preview the seek value chosen with the potentiometer.
Release S1	Apply the seek value chosen by the last position of the potentiometer.
Press S2	Apply the next instruction and remain paused.
Press S3	Stop and return to TRACK_SELECT.
Press S4	Resume timing the current instruction.

Table 9: Pushbuttons active while in the PAUSED state.

Seeking is available only for Track 4, so S1 must be ignored while any other track is paused.

6.3 Track Seeking

Seeking allows the user to jump directly to a chosen instruction rather than waiting for the track to reach it. It is available only for Track 4 while the music player is PAUSED. While S1 is held, the potentiometer selects the seek value, and this value is displayed as two hexadecimal digits on the display. Upon release, the music player must jump to the corresponding instruction and remain paused.

Track 4 contains exactly 2048 playable instructions. The seek value selects an instruction relative to the start of the track:

$$\text{instruction offset} = 8 \times \text{seek value}.$$

The 256 seek values therefore reach every eighth instruction of the track. Table 10 shows several examples.

Seek value	Display	Instruction offset	Position in Track 4
0x00 (0)	00	0	First instruction
0x40 (64)	40	512	One quarter
0x80 (128)	80	1024	Halfway
0xFF (255)	FF	2040	Near the end

Table 10: Examples of seek values and corresponding display and offset values, relative to the start of Track 4.

To correctly descramble an instruction after seeking, your program must also advance the LFSR to the appropriate state.

7 Serial Interface

Your program must implement a serial command parser, which allows the decoder, the descrambler, and the player to be exercised from a host computer. The serial interface must use 9600 baud, 8 data bits, 1 stop bit, and no parity.

7.1 Message Format

Every serial message consists of a start sequence, an identifier, and a payload, as shown in Table 11. The identifier selects the command and fixes the length of its payload.

Start sequence	Identifier	Payload
\	A single character	A fixed number of characters, possibly none

Table 11: Message format, consisting of a start sequence, identifier, and payload.

7.2 Commands

The various serial commands that can be specified are shown in Table 12.

Message	Payload	Action
\t	None	Decode and descramble <code>test_message</code> , and print the text.
\mc[32]	32 Base64 characters	Play the eight instructions encoded by <code>c[32]</code> .
\2, \3, \4	None	Action the press of the corresponding pushbutton.

Table 12: Serial commands.

A `\mc[32]` message is accepted only in TRACK SELECT, and must be rejected while a track is playing or paused. If S1 is held, the message must be acknowledged but have no effect. Its payload is encoded and scrambled in the same way as the encoded data, beginning from the initial LFSR state. Your program must decode and descramble it, play the eight instructions in order, and then return to TRACK SELECT with the selection unchanged. A terminating instruction ends playback early.

7.3 Responses

Every message must produce a response. If a message requires your program to print text, that text must begin with `?` and a space, as shown in Table 13.

Message	Printed text
<code>\t</code>	<code>? text</code>
<code>\mc[32]</code>	None
<code>\2, \3, \4</code>	None

Table 13: Serial messages and responses.

Every message must then be acknowledged, after any text it prints. An accepted message must be acknowledged with `#ACK`, and a rejected message with `#NACK`. Every line must end with the linefeed character `\n`. A message must be rejected when:

- an unknown identifier is supplied;
- any payload character is invalid for that identifier, noting that the entire payload must first be received before the message is rejected; or
- the action it requests is not available in the current state.

7.4 Examples

Examples of serial messages and responses are shown in Table 14.

	Message	Reason
Input	<code>\t</code>	
Output	<code>? Test message</code> <code>#ACK</code>	Accepted in every state.
Input	<code>\3</code>	
Output	<code>#ACK</code>	S3 is active in every state.
Input	<code>\mQjF2nUxpR7dGa0sPyV3bLmZ8tXcE9wQr</code>	
Output	<code>#ACK</code>	Valid payload, and in TRACK SELECT.
Input	<code>\mQjF2nUxpR7dGa0sPyV3bLmZ8tXcE9w*r</code>	
Output	<code>#NACK</code>	<code>*</code> is not a Base64 character.
Input	<code>\2</code>	
Output	<code>#NACK</code>	S2 is not active while a track is playing.

Table 14: Examples of serial messages and responses.

8 Hardware Configuration

8.1 Pushbuttons

All four pushbuttons must be debounced. When S1 is held, pressing other pushbuttons must have no effect.

8.2 Potentiometer

The ADC conversion result must be approximately proportional to the potentiometer position, across its full range of travel. Some representative values are shown in [Table 15](#).

Potentiometer position	ADC result
Fully counter-clockwise	0x00 (0)
Approximately halfway	0x80 (128)
Fully clockwise	0xFF (255)

Table 15: Examples of potentiometer positions and corresponding ADC results.

8.3 7-Segment Display

When displaying a hexadecimal value, the two-digit 7-segment display must be multiplexed with a refresh rate of 100 Hz, so that both digits appear to be continuously illuminated. During playback, the display must not be multiplexed, as each display byte drives the single digit it selects, as described in [Section 4.5.3](#). The segment patterns for hexadecimal digits are given in [Appendix A](#).

8.4 Buzzer

The buzzer must be driven by an approximately 50% duty-cycle square wave. This waveform must have a duty cycle of 0% when the buzzer is silent.

A Segment Patterns

Hexadecimal digits 0–F must appear on the 7-segment display as shown in [Figure 3](#).



Figure 3: Hexadecimal digits 0–F on the 7-segment display.